

Hive = Schema (metadata)+ Data
=> RDB + HDFS

//Metadata in metastore /data in HDFS or local

=====Hive CLI to run HiveQL =====

From home directory , type hive

hive

You will get hive prompt
hive>

-----Create hive database ,by default ,default database is selected .

//to list databases

show databases;

//to create new database

1. create database lti;

//to select database

use lti;

//Creating managed table

2.Creating a Table using Hive on the HDFS File.

create table patient(pid INT, pname STRING, drug STRING,gender string,
tot_amt INT) row format delimited fields terminated by ',' stored as textfile;

3.desc patient;

//Give detailed table information

4.desc extended patient;

5.## Loading data from local (ubuntu)

load data local inpath '/home/ak/datasets/patient.txt' into table patient;

-----first write the patient.txt to hdfs /demo folder -----

6. ## Loading data from HDFS

load data inpath '/demo/patient.txt' into table patient;

(This command will move the data from hdfs input location to hive table location)

7. ## using put command

```
hdfs dfs -put /home/ak/datasets/patient.dat /user/hive/warehouse/lti.db/patient/pat.dat
```

8. ## insert into command

```
Insert into table patient values (777,'aaa','vit','m',450);
```

duplicate table

8.create table patient1(pid INT, pname STRING, drug STRING,gender string, tot_amt INT) row format delimited fields terminated by ',' stored as textfile;

loading data from existing table

With truncate

```
insert overwrite table patient1 select * from patient where drug='Para';
```

Without truncate

```
Insert into table patient1 select * from patient where drug='Crocina';
```

Loading the result of the table to local directory(ubuntu)

```
insert overwrite local directory '/home/ak/hiveout/patout' stored as textfile select * from patient;
```

9. Execute the queries

```
Select * from patient;
```

```
select count(*) from patient;
```

```
select * from patient where drug='Para'
```

```
select sum(tot_amt) from patient;
```

```
select drug,sum(tot_amt) from patient group by drug;
```

```
insert overwrite local directory '/home/ak/results' select * from patient;
```

```
desc patient;
```

```
desc extended patient;
```

```
select patient.* from patient join drug on patient.drug=drug.drug;
```

```
select * from patient join drug on patient.drug=drug.drug;
```

Try the same join with both HDFS Files.

```
select p.* from patient p join drug1 d on p.drug=d.drug;
```

```
create table drug2(drug string) row format delimited fields terminated by ',' stored as  
textfile;
```

```
insert overwrite table drug2 select * from drug;
```

Left Outer Join:

```
select p.* from patient p left outer join drug1 d on p.drug=d.drug;
```

External Table:

```
// Creating just a schema on top of existing data (HDFS/local)
```

```
// Download the datasets folder to perform this operation
```

```
//write the /datasets/sample.txt from datasets folder to hdfs /iti/wordcount/ directory
```

```
hdfs dfs -put /home/ak/datasets/sample.txt /iti/wordcount/
```

```
## creating table on the top of existing HDFS file to access in hive
```

```
CREATE EXTERNAL TABLE wordcount(word string,count int)
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n'
```

```
STORED AS TEXTFILE LOCATION '/';
```

```
show tables;
```

```
Select * from wordcount;
```

```
## creating table on the top of existing local file to access in hive
```

```
CREATE EXTERNAL TABLE employee(empno int,ename string,sal int) ROW FORMAT  
DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
```

```
'file:///home/ak/datasets/empext/';
```

```
Select * from employee;
```

```
Show tables;
```

```
CREATE EXTERNAL TABLE dept(deptno int,dname string,Desg string) ROW FORMAT
DELIMITED FIELDS TERMINATED BY '|' STORED AS TEXTFILE LOCATION
'hdfs://localhost:9000/apr22';
```

=====

A. User defined location

```
CREATE EXTERNAL TABLE patext(pid int,pname string,drug string,gender string,amt int)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/lti/hiveext/';
```

```
## No directory is created in the table name 'patext' at '/lti/hiveext/00000_0
```

B.Default location

```
CREATE EXTERNAL TABLE patext1(pid int,pname string,drug string,gender string,amt int)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n'
STORED AS TEXTFILE ;
```

```
## Directory in the table name 'patext1' is created at '/user/hive/warehouse/lti.db/'
```

```
insert overwrite table patext select * from patient;
```

```
insert overwrite table patext1 select * from patient;
```

```
// When drop managed table , it deletes both metadata and data
// in external table , it deletes only metadata but data remains same
## dropping external and managed table
```

```
drop table patient1;
drop table patext;
drop table patext1;
```

=====File formats=====

```
//To see the difference in file size in hive , recommended to use large file at least in  mb
```

---To store as sequence file : (sequence file don't provide compression)

```
create table sequence_file(pid INT, pname STRING, drug STRING,gender string,
tot_amt INT) row format delimited fields terminated by ',' lines terminated by '\n' stored as
sequencefile;
```

```
insert into table sequence_file select * from newpatient;
```

---To store as rcf (Row Columnar file)

```
create table rcf_file(pid int,pname string,drug string,gender string,amt int) row format delimited
fields terminated by ',' lines terminated by '\n' stored as RCFile;
```

```
insert into table rcf_file select * from newpatient;
```

---To store as ORC file (Optimized Row Columnar)

```
create table orc_file(pid int,pname string,drug string,gender string,amt int) row format delimited
fields terminated by ',' lines terminated by '\n' stored as orcfile;
```

```
insert into table orc_file select * from patient;
```

// To store as parquet file (Columnar format / stored with metadata)

// In real time , parquet file is recommended

```
create table parquet_file(pid int,pname string,drug string,gender string,amt int) row format
delimited fields terminated by ',' lines terminated by '\n' stored as parquetfile;
```

```
insert overwrite table parquet_file select * from patient;
```

=====

Partition and bucketing

Patient → non-patiton

Partition:

#Creating partition using new column instead of existing table column

```
create table p_patient1(pid int,pname string,drug string,gender string,amt int)
```

```
partitioned by ( d string);
```

```
desc p_patient1;
```

```
## insert data into partition table
```

```
insert overwrite table p_patient1 partition(d='Paracetamol') select * from patient  
where drug='Para' ;
```

```
insert overwrite table p_patient1 partition(d='Painkiller') select * from patient  
where drug='metacin' or drug='Crocina';
```

```
##check the table data
```

```
select * from p_patient1;
```

```
-----  
#Creating partition using table column
```

```
create table p_patient2(pid int,pname string,gender string,amt int)  
partitioned by (drug string);
```

```
create table p_patient2(pid int,pname string,drug string,amt int)  
partitioned by (gender string);
```

```
desc p_patient2;
```

```
insert overwrite table p_patient2 partition(drug='Para')  
select pid,pname,gender,tot_amt from patient where drug ='Para';
```

```
insert overwrite table p_patient2 partition(gender='m')  
select pid,pname,drug,tot_amt from patient where gender ='m';
```

```
select * from p_patient2;
```

```
--querying non-partitioned table
```

```
Select drug,sum(tot_amt) from patient where drug='Para' group by drug;
```

```
--querying partitioned table
```

```
Select drug,sum(amt) from p_patient2 where drug='Para' group by drug;
```

dynamic partition :

set hive.exec.dynamic.partition.mode=nonstrict;

create table p_patientdy(pid int,pname string,amt int)
partitioned by (drug string,gender string);

insert overwrite table p_patientdy partition(drug,gender)
select pid,pname,tot_amt,drug,gender from patient ;

Bucketing (Default partition in mapreduce)

create table bucket_patient(pid int,pname string,drug string,gender string,amt int)
clustered by (drug) into 4 buckets;

hash value (clustered column) mod (no of buckets)

Hash value(Para) mod 4 --->

4000 mod 4 ⇒

insert overwrite table bucket_patient select * from patient;

select * from bucket_patient TABLESAMPLE(BUCKET 1 OUT OF 4 ON drug);

select * from bucket_patient TABLESAMPLE(50 percent);

select * from bucket_patient LIMIT 5;

create table bucket_patient4(pid int,pname string,drug string,gender string,amt int)
clustered by (drug) into 5 buckets;

insert overwrite table bucket_patient4 select * from patient;

create table part_buck_patient4(pid int,pname string,drug string,amt int) partitioned by (gender)
clustered by (drug) into 5 buckets;